

# Optimization Guide

v2 — Verified against 8 sources

For Jeremy's SGX Quant / Options / Research Pipeline Projects

|                  |                                                                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Sources verified | Anthropic Official Docs · HumanLayer Research · Florian Ultimate Guide · Awesome-Claude-Code · Anthropic Webinar (Mar 2026) |
| New in v2        | blocks · correct budget mechanics · context degradation thresholds · subagents & skills distinction · resource bookmarks    |
| Projects covered | Stock Scanner · Options Framework (Wheel/BPS) · Crypto Options · Research Pipeline                                          |
| Environment      | Claude Code (Cline/VS Code) · Python 3.11 · Streamlit · Google Sheets API · FMP API                                         |
| Version          | May 2026 · Claude Sonnet 4.6                                                                                                |

|          |                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE THIS | Read §1 for the honest assessment. §2 for architecture. §3 for writing rules. §4 for project templates. §5 for the update protocol (triggered by 'update CLAUDE.md'). §6 for advanced techniques. §7 for your resource bookmarks. |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# 1. Critical Assessment of the Source Article

The article you provided is directionally correct but has three factual errors and four significant omissions that matter for your use case. All findings below are verified against 8 sources including Anthropic's own documentation.

## 1.1 What holds up

| Claim                    | Verdict                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| ■ 150–200 line limit     | Confirmed by official docs AND HumanLayer empirical research. But the mechanism matters — see §1.2.           |
| ■ File hierarchy         | Confirmed: global → project → local → subdirectory, more specific overrides broader.                          |
| ■ Subdirectory CLAUDE.md | Confirmed: loads on-demand when Claude works in that directory.                                               |
| ■ HOW > WHAT > WHY       | Valid. Official docs: add when Claude 'makes the same mistake a second time.'                                 |
| ■ CLAUDE.md is advisory  | Confirmed. Official docs explicitly: 'Claude treats them as context, not enforced configuration.'             |
| ■ 12 hook events         | Accurate at time of writing. (May 2026 CHANGELOG now references 27 hook events across 5 categories — see §6.) |

## 1.2 What is wrong or misleading

|       |                                                                                                                                                                                             |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WRONG | The article measures the budget in 'instructions', not lines. This distinction is critical: a single instruction consuming 4 lines uses 4 lines of budget. Think in LINES, not item counts. |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|       |                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WRONG | The article says 'the system prompt uses ~50 of the 150–200 instructions.' Correct framing per HumanLayer's reverse-engineering: the system prompt contains ~50 individual INSTRUCTIONS (not lines). Since frontier LLMs reliably follow ~150–200 instructions total, your CLAUDE.md has roughly 100–150 instruction slots remaining — before you add rules, plugins, or skills. |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Source: HumanLayer blog: 'Writing a Good CLAUDE.md' (Nov 2025)

|       |                                                                                                                                                                                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WRONG | Context degradation threshold is stated as implicit. HumanLayer and DataCamp practitioners have independently measured: Claude output quality begins degrading at 20–40% context fill — far earlier than most assume. The practical advice to start /compact at 70% may be too late. |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Source: DataCamp: 'Claude Code Best Practices' (Mar 2026)

## 1.3 Critical omissions

|         |                                                                                                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MISSING | .claude/rules/ directory — the most important 2025 addition. Glob-scoped rule files that load only when relevant. Completely absent from the article. This is the primary solution to the budget problem. |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|         |                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------|
| MISSING | Auto Memory (MEMORY.md) — Claude's own persistent learning layer. First 200 lines load every session. Not mentioned at all. |
|---------|-----------------------------------------------------------------------------------------------------------------------------|

|         |                                                                                                                                                                            |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MISSING | conditional blocks — a March 2026 HumanLayer finding. Wrapping sections in XML condition tags dramatically improves adherence for task-specific rules. Not in the article. |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|         |                                                                                                                                                                                                                                                                   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MISSING | Skills vs CLAUDE.md distinction — Skills (.claude/skills/) are for complex multi-step domain knowledge that loads on demand. Official Anthropic docs explicitly say: if a procedure only matters sometimes, use a Skill, not CLAUDE.md. Article makes no mention. |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### MISSING

Code style rules belong in linters, not CLAUDE.md. HumanLayer: 'Never send an LLM to do a linter's job.' Style guidelines consume instruction budget and are inferior to deterministic tools. Instead, use hooks to run ruff/black after every write.

## 2. Official Architecture (May 2026)

### 2.1 Full memory hierarchy

| Priority       | Source                  | Lifetime     | Notes                                       |
|----------------|-------------------------|--------------|---------------------------------------------|
| 1<br>(highest) | System instructions     | Built-in     | Anthropic / operator — cannot override      |
| 2              | ~/.claude/CLAUDE.md     | Permanent    | Your global preferences across ALL projects |
| 3              | ./CLAUDE.md             | Permanent    | Project — commit to git, share with team    |
| 4              | ./claude/rules/*.md     | Conditional  | Glob-scoped rules — load only when matching |
| 5              | ./CLAUDE.local.md       | Permanent    | Personal project overrides — gitignore      |
| 6              | Auto memory (MEMORY.md) | Persistent   | Claude's own learnings — review monthly     |
| 7              | Conversation history    | Session only | Current session                             |
| 8 (lowest)     | Compacted summaries     | Session only | /compact output — lossy                     |

Source: [skillsplayground.com](https://skillsplayground.com) + Anthropic official docs ([code.claude.com/docs/en/memory](https://code.claude.com/docs/en/memory))

### 2.2 The instruction budget — precise mechanics

This is the most important thing to understand correctly:

| Parameter                                                        | Value                                             |
|------------------------------------------------------------------|---------------------------------------------------|
| Total instructions frontier LLM can reliably follow              | ~150–200                                          |
| Claude Code system prompt consumes                               | ~50 instructions                                  |
| Your usable budget (CLAUDE.md + rules + plugins + user messages) | ~100–150 instructions                             |
| HumanLayer's own CLAUDE.md target                                | Under 60 LINES                                    |
| Official Anthropic target                                        | Under 200 LINES per file                          |
| Context quality degradation begins                               | 20–40% context fill                               |
| Auto-compaction fires at                                         | ~83.5% context fill (lossy — can erase decisions) |
| Practical /compact trigger                                       | 60% capacity (not 70%)                            |

Source: HumanLayer (Nov 2025) · DataCamp (Mar 2026) · Anthropic docs

|                    |                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>KEY INSIGHT</b> | CLAUDE.md instructions get followed ~70% of the time. Hooks enforce rules at 100%. Treat CLAUDE.md as the place for guidance; hooks as the place for enforcement. |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 2.3 Extension mechanisms — when to use each

| Mechanism                 | When to use                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|
| CLAUDE.md                 | Permanent facts every session needs: build commands, architecture overview, non-obvious constraints. Target: under 80 lines. |
| ./claude/rules/*.md       | File-type or path-scoped rules. Load only when Claude works in matching files. Use for subsystem-specific conventions.       |
| Skills (./claude/skills/) | Complex, multi-step domain procedures. Load on demand via description match. Better than embedding procedures in CLAUDE.md.  |

|                             |                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Subagents (.claude/agents/) | Specialized assistants with isolated context and their own tool access. Use for parallel tasks or focus-intensive work.             |
| Hooks (settings.json)       | Deterministic enforcement. Runs regardless of Claude's judgment. Use for anything that <b>MUST</b> happen (linting, safety blocks). |
| Auto Memory (MEMORY.md)     | Claude's learned preferences. Audit monthly with /memory. Prune entries that duplicate your CLAUDE.md.                              |

Source: Anthropic official docs · Skills guide (websearchapi.ai)

### 3. Writing Principles (Evidence-Based, Multi-Source)

#### 3.1 The six rules with the strongest evidence

##### 1. Corrections only — never confirmations

Add a line only when Claude has actually made a mistake on your codebase. If Claude has never failed to use pandas correctly, don't write 'use pandas'. Official Anthropic docs: 'For each line, ask: would removing this cause Claude to make mistakes? If not, cut it.'

Source: Anthropic official best-practices page

##### 2. Think in lines, not items

Each line of markdown consumes one line of budget. A 3-word bullet wastes as much as a 15-word precise instruction. Reference files with @filename instead of embedding — they don't consume CLAUDE.md budget.

Source: Anthropic memory docs

##### 3. Specific and verifiable beats vague

'Use 2-space indentation' beats 'keep formatting clean'. 'Never use --force-push on main' beats 'be careful with git'. Claude follows instructions it can test against actual output.

Source: Anthropic official best-practices page

##### 4. Never put code style rules in CLAUDE.md

Use ruff, black, or similar deterministic tools. Add a PostToolUse hook to run them automatically after every file write. Style instructions consume budget and LLMs are slower and less reliable than linters.

Source: HumanLayer (Nov 2025): 'Never send an LLM to do a linter's job'

##### 5. Use tags for task-specific sections

Wrap sections that only apply sometimes in XML tags. Example: ... testing rules .... This dramatically improves adherence compared to flat bullet lists. Do NOT wrap foundational sections (project identity, stack, build commands) — those apply always.

Source: HumanLayer blog: 'Getting Claude to Actually Read Your CLAUDE.md' (Mar 2026)

##### 6. Move domain expertise to Skills, not CLAUDE.md

If a procedure is complex (multi-step) or only applies sometimes, build a Skill file. Skills load on demand via description matching — zero cost to your CLAUDE.md budget. Example: your options pricing calculation workflow → .claude/skills/options-pricing.md

Source: Anthropic official docs + Skills guide

#### 3.2 What to include vs exclude (Anthropic's own table)

| ■ Include                                  | ■ Exclude                                          |
|--------------------------------------------|----------------------------------------------------|
| Bash commands Claude can't guess           | Anything Claude can figure out by reading code     |
| Non-obvious constraints & gotchas          | Standard language conventions Claude already knows |
| Build, test, lint commands                 | Code style rules (use linters + hooks instead)     |
| Repository etiquette (branch naming, PRs)  | Detailed API documentation (link instead)          |
| Architecture decisions specific to project | Long explanations or tutorials (use Skills)        |
| Developer environment quirks (env vars)    | File-by-file descriptions of the codebase          |
| Known Claude failure patterns on YOUR code | Self-evident practices like 'write clean code'     |

Source: Anthropic official best-practices page (code.claude.com/docs/en/best-practices)

---

## 4. Project Templates — Copy and Adapt

These templates incorporate all principles from §3. They use the pattern, reference files rather than embedding, and stay well under 80 lines per file. Adapt paths and rules to your actual discovered failures.

### 4.1 Global `~/.claude/CLAUDE.md`

*Personal preferences across ALL projects. Keep this lean — it competes with every project's CLAUDE.md for instruction budget.*

```
# Jeremy — Global Preferences

## Identity
Quant/flow trader, Singapore (SGX equities + options). Python 3.11, Streamlit, Google Sheets API.
Primary tools: VS Code (Cline), Claude Code, MooMoo, TradingView.

## Communication
- Be direct. Lead conclusions before reasoning. Flag assumptions before proceeding.
- When uncertain, say so explicitly — do not hedge silently.

## Code Defaults (when no project CLAUDE.md overrides)
- Python 3.11+. Type hints on all function signatures.
- pathlib not os.path. f-strings not .format(). pd.concat() not df.append().
- Docstrings: one-line summary + Args/Returns for any function > 10 lines.

## Git
- Commit messages: imperative mood, <72 chars, conventional commits.
- Never force-push to main or master. Never commit .env or secrets.
```

### 4.2 Stock Scanner — `./CLAUDE.md`

*Streamlit + FMP API + Google Sheets + SQLite. Keep under 80 lines; move subsystem rules to rules/ files.*

```
# Stock Scanner — Project

## Stack
Python 3.11 | Streamlit | pandas | FMP API | Google Sheets API | SQLite cache
See @requirements.txt for deps. See @README.md for architecture.

## Build & Run
- Run: `streamlit run app.py`
- Test: `pytest tests/ -v`
- Lint: handled automatically by hook (ruff + black)

## Architecture
- app.py: Streamlit UI only — NO business logic in UI layer
- scanner/: core scanning logic
- data/: FMP API client + SQLite caching
- sheets/: Google Sheets write helpers
- See @ARCHITECTURE.md for module dependency graph

## SGX Rules
- Ticker format: SGX code (e.g. 'D05' for DBS), not Yahoo Finance format.
- Market hours: SGT (UTC+8). Open 09:00, close 17:00. Pre-market 08:30.
- Lot sizes: 100 shares. Flag any position sizing that ignores this.

<important if='you are working on FMP API or data fetching'>
- Rate limit: 150 calls/min on free tier. Always check headers before batch calls.
- Cache all responses in SQLite with TTL. Never re-fetch within TTL window.
- Exponential backoff on 429. Max 3 retries.
</important>

<important if='you are working on Google Sheets integration'>
```

```
See .claude/rules/google-sheets.md
</important>
```

```
## Known Failures (corrections only)
- st.cache_data NOT st.cache – old decorator is deprecated.
- pandas: NEVER chain .loc and .iloc. Use .loc[] for label-based indexing.
```

### 4.3 Options Framework — ./CLAUDE.md

```
# Options Strategy Framework
```

```
## Stack
```

```
Python 3.11 | pandas | numpy | scipy | MooMoo API | Binance API
See @requirements.txt | See @strategies/README.md for strategy inventory
```

```
## Build & Run
```

```
- Backtest: `python run_backtest.py --strategy wheel --ticker D05`
- Test: `pytest tests/ -v --tb=short`
- Lint: handled by hook
```

```
## Strategy Taxonomy
```

```
- Wheel Strategy: SGX equities (cash-secured puts + covered calls)
- Bull Put Spreads: SGX ETFs, defined-risk income
- Crypto Options: BTC/ETH on Binance – separate module (crypto/), different margin model
- Do NOT mix SGX and Binance position-sizing logic.
```

```
## Financial Rules (always apply)
```

```
- All P&L in SGD unless crypto (then USD). Label currency explicitly.
- Max position: 5% of notional per single ticker – hard rule, never override.
- Do NOT annualize returns without stating the assumption explicitly.
- Options premium: express as SGD per contract (100 shares), not per share.
```

```
<important if='you are working on options pricing or Greeks'>
See .claude/skills/options-pricing.md
</important>
```

```
<important if='you are working on backtesting or simulation'>
See .claude/rules/backtesting.md
</important>
```

```
## Known Failures
```

```
- Binance API: use async client for bulk fetches; sync for order execution only.
- MooMoo API: check session state before submission – market vs extended hours differ.
- Greeks: always recalculate at current spot. Never cache Greeks between price moves.
```

### 4.4 Research Pipeline — ./CLAUDE.md

```
# Research Pipeline
```

```
## Stack
```

```
Python 3.11 | FMP API | OpenRouter API | Google Sheets | pandas | SQLite
See @pipeline/README.md for data flow. See @config/api_limits.md for rate limits.
```

```
## Run
```

```
- `python pipeline.py --mode [daily|weekly|adhoc]`
- Test: `pytest tests/ -v`
```

```
## Data Rules (always apply)
```

```
- All dates: ISO 8601 (YYYY-MM-DD). All times: UTC + 'Z' suffix.
- Singapore timezone is UTC+8. Always convert to UTC for storage.
- Column naming: snake_case. No spaces in headers ever.
- Numerics: 2dp for prices, 4dp for ratios, 0dp for volumes.
```



<important if='you are working on FMP API calls'>

- Rate limit: 150 calls/min. Exponential backoff on 429, max 3 retries.
- Cache all responses in SQLite with TTL. Never re-fetch within TTL.
- Some FMP fields return None for SGX stocks – handle None explicitly.

</important>

<important if='you are working on OpenRouter or LLM calls'>

- Use claude-sonnet-4-6 for analysis; haiku-4-5 for classification tasks.
- Never hardcode model names – use constants from config/models.py.

</important>

## Known Failures

- Do NOT hardcode API keys – all secrets via environment variables or .env (gitignored).
- Google Sheets: batch\_update() ONLY. Never cell-by-cell writes in loops.

## 4.5 Rules files — .claude/rules/

*These load ONLY when Claude works with matching file patterns. Zero CLAUDE.md budget cost.*

### File: .claude/rules/google-sheets.md

```
---
globs: ['**/sheets*.py', '**/gsheets*.py', '**/google_sheets*.py']
---
# Google Sheets Rules
- Use gspread (NOT google-api-python-client directly).
- batch_update() for ALL multi-cell writes. Never cell-by-cell in a loop.
- Quota: 300 write requests/min. Add 0.2s delays for bulk operations.
- Auth: service account JSON from env var GOOGLE_SHEETS_CREDS.
- Reference sheets by ID, not name – names change, IDs don't.
```

### File: .claude/rules/backtesting.md

```
---
globs: ['**/backtest*.py', '**/simulate*.py']
---
# Backtesting Rules
- No look-ahead bias: signals must use only data available at bar close.
- Transaction costs: SGX = 0.28% commission + 0.1% clearing (min SGD 25).
- Slippage: 0.1% for illiquid SGX small-caps; 0.05% for STI components.
- Walk-forward validation required before any strategy deployment.
- Report: Sharpe, Calmar, Max DD, Win Rate, Avg Win/Loss. No cherry-picking.
```

## 4.6 Skills files — .claude/skills/

*Complex domain expertise that loads on demand. Better than embedding in CLAUDE.md.*

### File: .claude/skills/options-pricing.md

```
---
description: Use this skill when pricing options, calculating Greeks (delta/gamma/theta/vega),
or implementing Black-Scholes, Binomial trees, or IV calculations.
---
# Options Pricing Skill

## Model Selection
- European options: Black-Scholes via scipy.stats.norm.cdf (never implement CDF manually).
- American options: Binomial tree (Black-Scholes underprices early exercise).
- Barrier/exotic: consult Jeremy before proceeding.

## SGX-Specific Parameters
- Risk-free rate: 3-month SG T-bill rate from MAS data website (not US rates).
- IV validation:  $0 < IV < 3.0$  (300%). Reject and flag outliers before calculating.
- Delta hedge trigger: recalculate after every 1% spot move.

## Output Format
- Always return a dict with keys: delta, gamma, theta, vega, price, iv_rank.
- State units explicitly in docstring (per contract, per share, annualized/daily).
```

## 5. The 'update CLAUDE.md' Protocol

This section defines exactly what Claude Code should do when you type 'update CLAUDE.md'. Upload this PDF to Claude Code and reference Section 5 as your update protocol.

### TRIGGER

When Jeremy types 'update CLAUDE.md' (any casing, with or without project name), execute this protocol exactly.

### Step 1 — Read before writing

- Read the current ./CLAUDE.md (or the specified project's file).
- Read ~/.claude/projects/memory/MEMORY.md if it exists.
- Scan conversation history for: corrections Jeremy made, mistakes Claude repeated, clarifications Jeremy typed twice.

### Step 2 — Decision rules per candidate addition

| Condition                                           | Action                                                     |
|-----------------------------------------------------|------------------------------------------------------------|
| Claude made this mistake $\geq 2$ times in session  | ADD to CLAUDE.md or rules file                             |
| Jeremy corrected this $\geq 2$ times in chat        | ADD to CLAUDE.md                                           |
| Single one-off correction this session              | SKIP — add only after second occurrence                    |
| Instruction is file-type/path specific              | ADD to .claude/rules/.md with glob                         |
| Instruction is complex multi-step procedure         | CREATE or UPDATE .claude/skills/.md                        |
| Instruction is code style (indentation, formatting) | SKIP — ensure hook handles via linter                      |
| Instruction is personal preference not project rule | ADD to ~/.claude/CLAUDE.md instead                         |
| Root CLAUDE.md would exceed 80 lines                | Move oldest rule to rules/ file first                      |
| Behavior must be enforced (not just preferred)      | Use settings.json hook; note in CLAUDE.md                  |
| Entry duplicates something in MEMORY.md             | Keep in CLAUDE.md (higher priority); delete from MEMORY.md |

### Step 3 — audit

After adding any rule, check: does this rule apply to every session, or only sometimes? If only sometimes, wrap it in an block. Keep foundational rules (project identity, stack, build commands) unwrapped.

### Step 4 — Line budget check

#### HARD LIMIT

Root CLAUDE.md must stay under 80 lines (not 200 — leave headroom and reduce instruction competition). If this limit would be breached: (1) find the least-referenced existing rule, (2) move it to an appropriate rules/ or skills/ file, (3) then add the new rule.

### Step 5 — Report to Jeremy

## CLAUDE.md Update Summary

File: ./CLAUDE.md | Lines before/after:  $X \rightarrow Y$  (budget:  $Y/80$ )

Added:

- [rule] — reason: [why / how many times mistake occurred]

Modified:

- [old]  $\rightarrow$  [new] — reason: [what changed]

Skipped:

- [candidate] — reason: [one-off / style rule / belongs in rules/]

Moved to rules/ or skills/:

- [rule] → .claude/rules/[file].md (glob: [pattern])

MEMORY.md action:

- [deleted X entries that duplicated CLAUDE.md / promoted Y entries]

### Step 6 — Monthly Auto Memory audit

Once a month, run /memory in Claude Code. Look for: (a) entries duplicating CLAUDE.md — delete from memory, (b) entries worth promoting to CLAUDE.md — add formally, (c) stale entries from old project phases — delete. This prevents drift between explicit rules and Claude's learned behavior.

---

## 6. Advanced Techniques

### 6.1 Hooks — recommended setup for trading projects

*CLAUDE.md gives guidance (~70% adherence). Hooks give enforcement (100% adherence). Use hooks for anything non-negotiable.*

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [{"type": "command",
        "command": "cd $PROJECT_ROOT && ruff check . --fix --quiet && black . --quiet"}]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{"type": "command",
        "command": "python .claude/hooks/block_dangerous.py"}]
      }
    ],
    "Stop": [
      {
        "hooks": [{"type": "command",
        "command": "python .claude/hooks/session_summary.py"}]
      }
    ]
  }
}
```

**File: `.claude/hooks/block_dangerous.py`**

```
import json, sys

BLOCKLIST = [
  'rm -rf', 'DROP TABLE', 'DELETE FROM',
  'git push --force',
  'ORDER', # prevent live order in backtest context
  'api_key =', # prevent key logging
]

data = json.load(sys.stdin)
cmd = data.get('tool_input', {}).get('command', '')

for danger in BLOCKLIST:
  if danger.lower() in cmd.lower():
    print(f'BLOCKED: [{danger}]', file=sys.stderr)
    sys.exit(2) # sends error back to Claude as feedback
sys.exit(0)
```

### 6.2 Subagents for trading research workflows

Subagents run in isolated context with their own tools. Use them for parallel tasks (e.g., backtesting one strategy while researching another) or focus-intensive work (e.g., a dedicated security reviewer for your API code).

```
# .claude/agents/backtesting-agent.md
---
name: backtesting-agent
description: Runs backtests, validates strategies, checks for look-ahead bias.
Use when: implementing or validating any historical simulation.
tools: Read, Write, Bash
```

```
model: sonnet
---
You are a quantitative analyst specializing in backtesting.
You apply the rules in .claude/rules/backtesting.md strictly.
Always run pytest after implementing. Report Sharpe, Calmar, Max DD.
```

6.3 Full hook event reference (May 2026 — 12 primary events)

| Event              | When                      | Primary use case                         |
|--------------------|---------------------------|------------------------------------------|
| PreToolUse         | Before any tool call      | Block dangerous commands, inject context |
| PostToolUse        | After any tool call       | Auto-lint, auto-format, auto-test        |
| PostToolUseFailure | On tool error             | Log failures, alert                      |
| PermissionRequest  | Claude asks permission    | Route approval decisions                 |
| UserPromptSubmit   | When you send a message   | Validate inputs, log                     |
| Stop               | Session ends              | Summary, notifications                   |
| SubagentStop       | A subagent finishes       | Chain next agent in pipeline             |
| SubagentStart      | A subagent starts         | Inject context, log                      |
| PreCompact         | Before context compaction | Preserve critical state                  |
| SessionStart       | Session opens             | Load dynamic context                     |
| SessionEnd         | Session closes            | Archive, report                          |
| Notification       | Permission prompts, idle  | Custom TTS, Slack alerts                 |

---

## 7. Curated Resource Bookmarks

All 10 resources reviewed and assessed. Ranked by relevance to your use case.



### Anthropic Official Memory Docs

[code.claude.com/docs/en/memory](https://code.claude.com/docs/en/memory)

The canonical source for CLAUDE.md architecture, rules/, auto memory, and hierarchy. Updated continuously. Read this first.



### Anthropic Official Best Practices

[code.claude.com/docs/en/best-practices](https://code.claude.com/docs/en/best-practices)

The clearest official guide: include/exclude table, verification patterns, Plan Mode workflow, subagent setup. High signal.



### HumanLayer: Writing a Good CLAUDE.md

[humanlayer.dev/blog/writing-a-good-claude-md](https://humanlayer.dev/blog/writing-a-good-claude-md)

The most rigorous empirical analysis of instruction budgets. Reverse-engineered the system prompt (~50 instructions). Validated the 150–200 total limit. Essential reading.



### HumanLayer: Getting Claude to Read Your CLAUDE.md

[humanlayer.dev/blog/stop-claude-from-ignoring-your-claude-md](https://humanlayer.dev/blog/stop-claude-from-ignoring-your-claude-md)

March 2026. The source for the conditional block technique. Empirically validated improvement in adherence. High priority.



### Claude Code Best Practice on GitHub

[github.com/shanraishan/claude-code-best-practice](https://github.com/shanraishan/claude-code-best-practice)

19.7k stars. Community-compiled. Good for cross-referencing patterns and CLAUDE.md examples. Includes memory system deep-dive.



### Anthropic Advanced Patterns Webinar

[anthropic.com/webinars/claude-code-advanced-patterns](https://anthropic.com/webinars/claude-code-advanced-patterns)

March 2026. Subagents as orchestrators, MCP at scale, CLAUDE.md for monorepos, CI/CD patterns. Directly relevant to your multi-module trading setup.



### Awesome-Claude-Code (hesreallyhim)

[github.com/hesreallyhim/awesome-claude-code](https://github.com/hesreallyhim/awesome-claude-code)

Curated list of skills, hooks, slash-commands, agent orchestrators. Use the /resources/claude.md-files directory for real-world CLAUDE.md examples to benchmark against.



### Claude Code Ultimate Guide (FlorianBruniaux)

[github.com/FlorianBruniaux/claude-code-ultimate-guide](https://github.com/FlorianBruniaux/claude-code-ultimate-guide)

Comprehensive beginner-to-power-user coverage with quizzes and cheatsheet. Good for team onboarding. Note: includes some security findings (655 malicious skills in supply chain — vet MCP servers before use).



### ClaudeLog

[claude-log.com](https://claude-log.com)

Independent community experiments and configuration guides. Good for practitioner-level observations about what actually works. Not official — treat as hypothesis generation.



### Claude Agent SDK Tutorial

[letsdatascience.com/blog/claude-agent-sdk-tutorial](https://letsdatascience.com/blog/claude-agent-sdk-tutorial)

Relevant if you want to build programmatic agents (e.g., automated pipeline that uses Claude Code). Lower priority until you need headless/non-interactive sessions.

## 8. Quick Reference Card

### Commands

| Command               | Purpose                                                                            |
|-----------------------|------------------------------------------------------------------------------------|
| /init                 | Generate initial CLAUDE.md from codebase scan (use --interactive for guided setup) |
| /memory               | View and edit auto memory (MEMORY.md) — run monthly                                |
| /compact              | Compress context — trigger at 60% fill, not 70%                                    |
| update CLAUDE.md      | Jeremy's trigger phrase — executes \$5 protocol                                    |
| @filename             | Reference a file without embedding (doesn't consume CLAUDE.md budget)              |
| /clear                | Mandatory at 90%+ context — always update CLAUDE.md first                          |
| Plan Mode (Shift+Tab) | Explore and plan before coding — reduces wrong-problem solutions                   |
| /memory edit          | Edit auto memory entries directly                                                  |

### Context window thresholds (revised based on empirical data)

| Fill % | Status   | Action             | Notes                                       |
|--------|----------|--------------------|---------------------------------------------|
| 0–40%  | GREEN    | Work freely        | Normal operation                            |
| 40–60% | YELLOW   | Monitor            | Note decisions made; be explicit in prompts |
| 60–70% | AMBER    | /compact now       | Before context degrades further             |
| 70–83% | RED      | Stop new work      | Only /compact or /clear                     |
| 83.5%+ | CRITICAL | Auto-compact fires | Lossy — may erase decisions; start fresh    |

Source: DataCamp (Mar 2026): degradation begins 20–40%. Auto-compact at ~83.5% confirmed by practitioners.

### Decision tree: where does this instruction go?

| Situation                                     | Correct location                             |
|-----------------------------------------------|----------------------------------------------|
| Applies to ALL your projects, personal style  | ~/.claude/CLAUDE.md                          |
| Applies to one project, share with team       | ./CLAUDE.md or ~/.claude/CLAUDE.md           |
| Applies to one project, personal only         | ./CLAUDE.local.md (gitignore)                |
| Applies to specific file types or directories | `.claude/rules/.md` with globs               |
| Complex multi-step domain procedure           | `.claude/skills/.md` with description        |
| Specialized assistant with isolated context   | `.claude/agents/.md` with YAML               |
| Must always happen, non-negotiable            | `.claude/settings.json` hooks                |
| Code style enforcement                        | Linters + PostToolUse hook (never CLAUDE.md) |
| Claude learned it from corrections            | Auto memory MEMORY.md (audit monthly)        |

Sources: Anthropic Official Docs (code.claude.com) · HumanLayer Research (Nov 2025, Mar 2026) · DataCamp Claude Code Best Practices (Mar 2026) · shanraishan/claude-code-best-practice (19.7k★) · hesreallyhim/awesome-claude-code · FlorianBruniaux/claude-code-ultimate-guide · Anthropic Advanced Patterns Webinar (Mar 2026) · VILA-Lab arXiv:2604.14228 · Releasebot/Anthropic CHANGELOG (May 2026). Generated May 2026.